SoC1&2: Tool Chain

RWU

Andreas Siggelkow
andreas.siggelkow@rwu.de

Elektrotechnik und Informatik

November 12, 2026

Outline

Tools

Project Setup

Single Test Case

Single Test Case with xsim

Regression

Pitfalls

Q&A

Tools

Tools

Tools

Installation: RISC-V GNU Toolchain

- Assembler, linker, objcopy
- Follow the current instructions on https://github.com/riscv/riscv-gnu-toolchain
- Recommended install target: `/opt/riscv`
- `make linux` builds the `riscv64-unknown-linux-gnu-*` tools, the prefix our CMake files use

```
git clone https://github.com/riscv/riscv-gnu-toolchain
cd riscv-gnu-toolchain
./configure --prefix=/opt/riscv
make linux
```

Tools

Installation: Caution with Vivado

- **Caution:** AMD Vivado ships its own RISC-V toolchain, which does not suit our needs. It may still be in your PATH (after sourcing `settings64.sh`).
- Our CMake files therefore call the tools with an absolute path:
  `/opt/riscv/bin/riscv64-unknown-linux-gnu-{as,ld,objcopy}`

Tools

Installation: Simulator and Build System

- Simulator (one of them is enough)
  - AMD Vivado simulator (xsim): `xvlog` / `xelab` / `xsim`, waveforms in the Vivado GUI
  - Verilator $\geq$ 5.0 (needed for `--timing`) plus GTKWave to view the FST waveform
- Build system
  - CMake $\geq$ 3.20 (plus make or ninja as the backend)

Project Setup

Project Setup

Project Setup

Project Layout (RWU_RV/RV_NoPipeline)

`rv64Sim/`	assembler test programs (`*.asm`) + `CMakeLists.txt` (ASM → MEM)
`src/`	RTL of the CPU and the top level
`../../ip_*/`	shared IPs (GPIO, UART, JTAG, CGU, DMem, ICU, trap CSR, ...)
`tb/`	testbenches (`tb_rv64i_instr.sv` = shared instruction testbench)
`sim/`	`CMakeLists.txt` (compile + simulate), `sim_main.cpp`, `*.tcl`, `*.wcfg`
`CMakeLists.txt`	top level: list of tests, testbench selection, regression

Project Setup

Three CMake Levels

```
CMakeLists.txt --> rv64Sim/CMakeLists.txt (target "mem")
               --> sim/CMakeLists.txt     (target "simulate"/"waves")
```

- The top level only orchestrates: one target per test
- `rv64Sim`: assembler program → memory image `rv64i.mem`
- `sim`: compile the RTL and run the simulation with that image

Single Test Case

Single test case: simulation with Verilator and waveforms

Single Test Case

Preparation

```
cd RWU_RV/RV_NoPipeline
rm -rf build
cmake -S . -B build -DSIMULATOR=verilator -DWAVES=ON
cmake --build build --target sim_instr38ld
```

- `rm -rf build`: start from a clean build directory
- Two steps: *configure* (`cmake -S ... -B ...`), then *build* (`cmake --build ...`)

Single Test Case

What the Options Mean

`-S .`	source directory (the dot means the current directory)
`-B build`	build directory; it is created if needed and holds all generated files
`-DSIMULATOR=verilator`	`-D` defines a CMake variable: simulator backend (`xsim` = default | `verilator`)
`-DWAVES=ON`	open the waveform viewer afterwards (single tests only, never together with `regression`)
`--build build`	runs the build in the directory `build`
`--target sim_instr38ld`	picks one test case: `sim_<name of the .asm file>`

Single Test Case

Under the Hood: What CMake Does

- The top-level `CMakeLists.txt` defines one target `sim_<test>` per test, each with its own build directory (here `build/instr38ld/`):

```
set(TEST_BUILD_DIR ${CMAKE_BINARY_DIR}/${TEST})
```

- Testbench selection: `instr38ld` is listed in `INSTR_TB_CONFIG`

```
"instr38ld"         "ld"          "38"
```

  → `tb/tb_rv64i_instr.sv` with `TEST_NAME=ld`, `EXPECTED=38` (GPIO value that signals a passing test)

- Tests that are not in that list use their own testbench `tb/tb_rv64i_<test>.sv`

Single Test Case

Under the Hood: What CMake Does

- Step 1: `build/instr38ld/asm/` is configured with `rv64Sim/CMakeLists.txt`;
  the target `mem` produces `rv64i.mem`
- Step 2: `build/instr38ld/sim/` is configured with `sim/CMakeLists.txt`;
  the target `waves` (`WAVES=ON`) or `simulate` (`WAVES=OFF`) runs
- Step 2 depends on `rv64i.mem`, so CMake always runs step 1 first

Single Test Case

The Same Steps by Hand (no CMake)

Shell variables used on the next slides (start in `RWU_RV/RV_NoPipeline`):

```
P=$PWD              # project root
IP=$P/../..         # IP root (ip_* directories)
RV=/opt/riscv/bin/riscv64-unknown-linux-gnu
mkdir -p build/instr38ld/asm build/instr38ld/sim
```

A) Assembler program → memory image

B) Verilate and compile the C++ model

C) Simulate

D) Look at the waveforms

Single Test Case

a) Assembler Program → Memory Image (1/2)

```
cd build/instr38ld/asm
$RV-as -march=rv64i_zicsr -mlittle-endian -g -o rv64i.o \
       $P/rv64Sim/instr38ld.asm -al=rv64i.lst
$RV-ld -b elf64-littleriscv -o rv64i.elf rv64i.o
```

- `as`: assemble; `-al=rv64i.lst` also writes a listing. CMake does not produce it, but it is handy for debugging.
- `ld`: link; an optional linker script can be passed with `-T <file>`, in CMake with `-DLINKER_SCRIPT=<file>`

Single Test Case

a) Assembler Program → Memory Image (2/2)

```
$RV-objcopy -O verilog --verilog-data-width 4 rv64i.elf rv64i.mem
sed -i 's/ /\n/g' rv64i.mem   # one word per line
sed -i 's/\r//g'  rv64i.mem   # remove ^M (CR), if present
sed -i '1d'       rv64i.mem   # delete the address header
```

- `objcopy`: ELF → Verilog hex format, one 32-bit word = one instruction
- The linker places `.text` at `0x100B0`; the header holds the word address: `@0000402C` (`0x100B0/4 = 0x402C`)
- Without `sed '1d'`, `$readmemh` would load the program there instead of at address 0, where the CPU starts. Required!

Single Test Case

b) Verilate and Compile the C++ Model

```
cd ../sim
verilator --cc --exe $P/sim/sim_main.cpp --build --sv --timing \
  -Wall -Wno-fatal -Wno-DECLFILENAME -Wno-PINCONNECTEMPTY -Wno-IMPORTSTAR \
  --Mdir obj_dir -DLOGIC_SV --trace-fst --top-module tb_rv64i \
  $P/sim/tb_waivers.vlt \
  $P/src/as_pack.sv  $P/src/asRegFile.sv \
  $IP/ip_dmem/src/{asDMemBack,asDMemFront,asDMemCore,asDMem,asDMemTop}.sv \
  $P/src/asIMem.sv  $P/src/asAlu.sv  $P/src/asAluBr.sv  $P/src/asInstrDecode.sv \
  $IP/ip_jtag/src/{tap_fsm,ir_cell,dr_cell,scan_cell,by_cell,ir_reg,dr_reg,ir_decode,jtag}.sv \
  $P/src/asMBpi.sv  $P/src/asSBpi.sv \
  $IP/ip_gpio/src/asGpio.sv  $IP/ip_irqper/src/as_irq_peripheral.sv \
  $IP/ip_gpio/src/asGpioTop.sv \
  $IP/ip_trapcsr/src/cc_trap_csr.sv  $P/src/asCPUx.sv  $P/src/asDecode.sv \
  $IP/ip_cgu/src/{asCguCore,asCguTop}.sv \
  $IP/ip_fifo/src/as_fifo.sv  $IP/ip_uart_v2/src/as_uart_top.sv \
  $IP/ip_icu/src/{cc_icu,cc_icu_top}.sv \
  $P/src/asTopMem.sv \
  $P/tb/tb_rv64i_instr.sv
```

Single Test Case

b) Verilator Options

`--cc --exe ... --build`	generate C++ from the RTL, compile it together with `sim_main.cpp` into `obj_dir/Vtb_rv64i`
`--timing`	supports the #delays in the testbench
`--trace-fst`	`sim_main.cpp` writes the waveform to `dump.fst`
`-DLOGIC_SV`	same define as for xsim (`xvlog -d LOGIC_SV`)

- The order of the files matters: `as_pack.sv` comes first, the testbench last
- The source list is in `sim/CMakeLists.txt` (`SOURCES_SV`)

Single Test Case

c) Simulate

```
cp ../asm/rv64i.mem riscvtest.mem
./obj_dir/Vtb_rv64i +TEST_NAME=ld +EXPECTED=38
```

- The file name is fixed: the testbench and `asIMem.sv` read `riscvtest.mem` with `$readmemh` from the current working directory
- `+TEST_NAME`, `+EXPECTED`: plusargs for `tb_rv64i_instr.sv`
- CMake does the same with:

```
cmake -E copy .../build/instr38ld/asm/rv64i.mem riscvtest.mem
cmake -P .../build/instr38ld/sim/run_sim.cmake
```

Single Test Case

c) Expected Output and Evaluation

```
CS #1: gpio=0x26  expected=0x26
Simulation ld succeeded
%Error: .../tb/tb_rv64i_instr.sv:74: Verilog $stop
Aborting...
```

`error.txt: 11/10/26 - ld:  Test ok`

- The testbench checks the GPIO value at every CS pulse:
  - GPIO == `EXPECTED` → "Simulation ld succeeded", "Test ok" in `error.txt`
  - no match after 20 CS pulses → "Test fail" in `error.txt`
- The test ends with `$stop`, so Verilator exits with code 1. That is not an error; `execute_process` in `run_sim.cmake` ignores it.
- `error.txt` is opened with `"a"`: results accumulate over runs!

Single Test Case

d) Look at the Waveforms

```
gtkwave dump.fst &
```

- With `-DWAVES=ON`, CMake starts GTKWave the same way after the simulation

Single Test Case

What CMake Prints: Configure

```
cmake -S . -B build -DSIMULATOR=verilator -DWAVES=ON
```

```
-- AS-TOP: WAVES=ON: Waveform GUI aktiv - nicht mit 'regression' kombinieren!
-- AS-TOP: SIMULATOR = verilator
-- AS-TOP: SIM_TARGET = waves
-- Configuring done (0.0s)
-- Generating done (0.0s)
-- Build files have been written to: .../RWU_RV/RV_NoPipeline/build
```

- The `AS-*` lines are our own messages from the `CMakeLists.txt` files

Single Test Case

What CMake Prints: Step 1, Configure the Assembler

```
cmake --build build --target sim_instr38ld
```

```
AS-TOP: Assembling instr38ld
 : cmake -E make_directory .../RWU_RV/RV_NoPipeline/build/instr38ld/asm
 : cmake -DASM_FILE=.../RWU_RV/RV_NoPipeline/rv64Sim/instr38ld.asm
 -S .../RWU_RV/RV_NoPipeline/rv64Sim -B .../RWU_RV/RV_NoPipeline/build/instr38ld/asm
 : cmake --build .../RWU_RV/RV_NoPipeline/build/instr38ld/asm --target mem
-- The ASM compiler identification is GNU
-- Found assembler: /usr/bin/cc
-- Configuring done (0.0s)
-- Generating done (0.0s)
-- Build files have been written to: .../RWU_RV/RV_NoPipeline/build/instr38ld/asm
```

- `/usr/bin/cc` is only CMake's language detection. The actual calls use `/opt/riscv/bin/...` (next slides).

Single Test Case

What CMake Prints: Step 1, Assemble and Link

```
AS-rv64Sim: Assembling .../RWU_RV/RV_NoPipeline/rv64Sim/instr38ld.asm
 : /opt/riscv/bin/riscv64-unknown-linux-gnu-as
 -march=rv64i_zicsr
 -mlittle-endian
 -g
 -o .../RWU_RV/RV_NoPipeline/build/instr38ld/asm/rv64i.o
 .../RWU_RV/RV_NoPipeline/rv64Sim/instr38ld.asm

AS-rv64Sim: Linking ELF
 : /opt/riscv/bin/riscv64-unknown-linux-gnu-ld
 -b elf64-littleriscv
 -o .../RWU_RV/RV_NoPipeline/build/instr38ld/asm/rv64i.elf
 .../RWU_RV/RV_NoPipeline/build/instr38ld/asm/rv64i.o
```

Single Test Case

What CMake Prints: Step 1, Memory Image

```
AS-rv64Sim: Generating Verilog MEM
 : /opt/riscv/bin/riscv64-unknown-linux-gnu-objcopy
 -O verilog
 --verilog-data-width 4
 .../RWU_RV/RV_NoPipeline/build/instr38ld/asm/rv64i.elf
 .../RWU_RV/RV_NoPipeline/build/instr38ld/asm/rv64i.mem
 : sed -i 's/ /\n/g' .../RWU_RV/RV_NoPipeline/build/instr38ld/asm/rv64i.mem
 : sed -i 's/\r//g' .../RWU_RV/RV_NoPipeline/build/instr38ld/asm/rv64i.mem
 : sed -i '1d' .../RWU_RV/RV_NoPipeline/build/instr38ld/asm/rv64i.mem
[100%] Built target mem
```

Single Test Case

What CMake Prints: Step 2, Configure the Simulation

```
AS-TOP: Simulating instr38ld [verilator]
 : /usr/bin/cmake
 -DPROJECT_ROOT=.../RWU_RV/RV_NoPipeline
 -DIP_ROOT=.../RWU_RV/RV_NoPipeline/../..
 -DRV64_MEM=.../RWU_RV/RV_NoPipeline/build/instr38ld/asm/rv64i.mem
 -DTB_FILE=.../RWU_RV/RV_NoPipeline/tb/tb_rv64i_instr.sv
 -DSIMULATOR=verilator
 -DTB_TEST_NAME=ld;-DTB_EXPECTED=38
 -S .../RWU_RV/RV_NoPipeline/sim
 -B .../RWU_RV/RV_NoPipeline/build/instr38ld/sim
 : /usr/bin/cmake --build .../RWU_RV/RV_NoPipeline/build/instr38ld/sim --target waves
-- Configuring done (0.0s)
-- Generating done (0.0s)
-- Build files have been written to: .../RWU_RV/RV_NoPipeline/build/instr38ld/sim
```

- The ; in `-DTB_TEST_NAME=ld;-DTB_EXPECTED=38` is only how CMake prints a list: these are two separate arguments

Single Test Case

What CMake Prints: Step 2, Verilate (1/2)

```
AS-sim: Verilating and building tb_rv64i
 : verilator
 --cc
 --exe .../RWU_RV/RV_NoPipeline/sim/sim_main.cpp
 --build
 --sv
 --timing
 -Wall
 -Wno-fatal
 -Wno-DECLFILENAME
 -Wno-PINCONNECTEMPTY
 -Wno-IMPORTSTAR
 --Mdir .../RWU_RV/RV_NoPipeline/build/instr38ld/sim/obj_dir
 -DLOGIC_SV
 --trace-fst
 --top-module tb_rv64i
 .../RWU_RV/RV_NoPipeline/sim/tb_waivers.vlt
 (source list: next slide)
```

Single Test Case

What CMake Prints: Step 2, Verilate (2/2)

`SOURCES_SV`, printed as one `;`-separated list (`./` = project root, `.../` = IP root):

```
./src/as_pack.sv;./src/asRegFile.sv;
.../ip_dmem/src/asDMemBack.sv;.../ip_dmem/src/asDMemFront.sv;
.../ip_dmem/src/asDMemCore.sv;.../ip_dmem/src/asDMem.sv;.../ip_dmem/src/asDMemTop.sv;
./src/asIMem.sv;./src/asAlu.sv;./src/asAluBr.sv;./src/asInstrDecode.sv;
.../ip_jtag/src/tap_fsm.sv;.../ip_jtag/src/ir_cell.sv;.../ip_jtag/src/dr_cell.sv;
.../ip_jtag/src/scan_cell.sv;.../ip_jtag/src/by_cell.sv;.../ip_jtag/src/ir_reg.sv;
.../ip_jtag/src/dr_reg.sv;.../ip_jtag/src/ir_decode.sv;.../ip_jtag/src/jtag.sv;
./src/asMBpi.sv;./src/asSBpi.sv;
.../ip_gpio/src/asGpio.sv;.../ip_irqper/src/as_irq_peripheral.sv;
.../ip_gpio/src/asGpioTop.sv;
.../ip_trapcsr/src/cc_trap_csr.sv;./src/asCPUx.sv;./src/asDecode.sv;
.../ip_cgu/src/asCguCore.sv;.../ip_cgu/src/asCguTop.sv;
.../ip_fifo/src/as_fifo.sv;.../ip_uart_v2/src/as_uart_top.sv;
.../ip_icu/src/cc_icu.sv;.../ip_icu/src/cc_icu_top.sv;
./src/asTopMem.sv;./tb/tb_rv64i_instr.sv
```

Single Test Case

What CMake Prints: Step 2, Simulate and Waves

```
AS-sim: Running Verilator simulation + GTKWave
 : cmake -E copy .../RWU_RV/RV_NoPipeline/build/instr38ld/asm/rv64i.mem riscvtest.mem
 : cmake -P .../RWU_RV/RV_NoPipeline/build/instr38ld/sim/run_sim.cmake
 : gtkwave dump.fst &
CS #1: gpio=0x26  expected=0x26
Simulation ld succeeded
%Error: .../RWU_RV/RV_NoPipeline/tb/tb_rv64i_instr.sv:74: Verilog $stop
Aborting...
```

- Same result as by hand: compare with “c) Expected Output”

Single Test Case with xsim

Variant: the same single test with xsim (Vivado)

Single Test Case with xsim

Preparation

```
source <Vivado install dir>/settings64.sh
cd RWU_RV/RV_NoPipeline
rm -rf build
cmake -S . -B build -DSIMULATOR=xsim -DWAVES=ON
cmake --build build --target sim_instr38ld
```

- `settings64.sh` puts `xvlog`, `xelab` and `xsim` into the PATH
- `xsim` is the default: `-DSIMULATOR=xsim` may be left out
- Step 1 (assembler → `rv64i.mem`) is identical to Verilator; only step 2 in `sim/CMakeLists.txt` differs

Single Test Case with xsim

Under the Hood: Three xsim Steps

`xvlog` *compile*: parses all SystemVerilog sources into the library `xsim.dir`

`xelab` *elaborate*: builds the design hierarchy below `tb_rv64i` and creates the snapshot `tb_rv64i_snapshot`

`xsim` *simulate*: runs the snapshot, with or without the Vivado GUI

- CMake targets in `sim/CMakeLists.txt`: `compile` → `elaborate` → `simulate` (`WAVES=OFF`) or `waves` (`WAVES=ON`)
- Each step leaves a stamp file (`compile.stamp`, `elaborate.stamp`); CMake reruns a step only if the stamp is missing or older than its input

Single Test Case with xsim

The Same Steps by Hand: Compile and Elaborate

In `build/instr38ld/sim`, with `P` and `IP` as before:

```
xvlog --sv --incr --relax -d LOGIC_SV <sources>
cp ../asm/rv64i.mem riscvtest.mem
xelab -debug all -top tb_rv64i -snapshot tb_rv64i_snapshot
```

`<sources>`	the same file list (and order) as for Verilator, without `tb_waivers.vlt`
`--sv`	treat the files as SystemVerilog
`--incr`	incremental: only recompile files that changed
`--relax`	relaxed language checks
`-d LOGIC_SV`	the same define as `-DLOGIC_SV` for Verilator
`-debug all`	keep all signals visible for the waveform viewer

Single Test Case with xsim

The Same Steps by Hand: Simulate

With the Vivado GUI and waveforms (`WAVES=ON`):

```
xsim --gui tb_rv64i_snapshot \
     --testplusarg "TEST_NAME=ld" --testplusarg "EXPECTED=38" \
     --tclbatch $P/sim/xsim_cfg2.tcl
```

Without the GUI (`WAVES=OFF`):

```
xsim tb_rv64i_snapshot \
     --testplusarg "TEST_NAME=ld" --testplusarg "EXPECTED=38" \
     --tclbatch $P/sim/xsim_cfg.tcl
```

- `--testplusarg` passes the same plusargs as `+TEST_NAME` / `+EXPECTED` for Verilator

Single Test Case with xsim

The Tcl Scripts

`sim/xsim_cfg.tcl` (batch)

```
log_wave -recursive * -v
run all
exit
```

`sim/xsim_cfg2.tcl` (GUI)

```
log_wave -recursive *
add_wave {{*}}
run all
#exit
```

- `run all` runs until the testbench's `$stop`
- Batch: `exit` ends xsim afterwards
- GUI: no `exit`, so the Vivado window stays open with all top-level signals in the waveform; add more signals from the scope tree (or load one of the `sim/*.wcfg` files)
- The result is evaluated as with Verilator: console output and `error.txt` in `build/instr38ld/sim`

Regression

Regression: all tests at once

Regression

Run All Tests

```
cd RWU_RV/RV_NoPipeline
rm -rf build
cmake -S . -B build -DWAVES=OFF          # xsim (default)
cmake --build build --target collect_errors -j8
```

- With Verilator: add `-DSIMULATOR=verilator` as for a single test
- `-DWAVES=OFF` is required: otherwise every test opens a waveform viewer
- `-j8`: up to 8 tests run in parallel. That is safe, because every test has its own directory `build/<test>/`
- Result: `regression.txt` in the project root

Regression

What Happens

```
collect_errors
  `-- depends on: regression
        `-- depends on: sim_<test>   for every test in ASM_TESTS
              (each one exactly as in "Single Test Case")
  then: cmake -P collect_errors.cmake
        build/<test>/sim/error.txt  -->  regression.txt
```

- `ASM_TESTS` in the top-level `CMakeLists.txt` lists all tests (currently 64)
- `regression` alone only runs the tests; `collect_errors` also writes the report
- `regression.txt` is rewritten on every run; the `error.txt` files are not (see Pitfalls)

Regression

Reading the Report

```
=== RV64I REGRESSION REPORT ===

--- instr99scantest ---
11/06/26 - scan test: Test ok

--- instr01loadbyte ---
11/06/26 - load_byte: Test ok

--- instr38ld ---
11/06/26 - ld: Test ok
...
```

One section per test with the content of its `error.txt`:

- `Test ok`: passed
- `Test fail`: failed
- `MISSING error.txt`: the test did not report at all. Treat it as a failure!

Quick check:

```
grep -c "Test ok" regression.txt                    # 64 expected
grep -B1 "fail\|MISSING" regression.txt               # which tests?
```

Regression

Adding a New Test

1. Write the program `rv64Sim/<test>.asm`
2. Add `<test>` to `ASM_TESTS` in the top-level `CMakeLists.txt`
3. Choose the testbench:
   - shared instruction testbench: add a line to `INSTR_TB_CONFIG`

     ```
     "<test>"   "<name in error.txt>"   "<expected GPIO value>"
     ```

   - otherwise: write your own `tb/tb_rv64i_<test>.sv`
4. Configure again, then test it alone first:

   ```
   cmake -S . -B build
   cmake --build build --target sim_<test>
   ```

Pitfalls

Toolchain and Cache Variables

- **Vivado's RISC-V toolchain in the PATH**
  After `source settings64.sh`, `riscv64-unknown-*` may resolve to Vivado's tools. Always call `/opt/riscv/bin/...` with the absolute path.
- **`WAVES` and `SIMULATOR` are cache variables**
  They stay set in `build/CMakeCache.txt` until you change them explicitly or delete `build/`:

```
cmake -S . -B build -DWAVES=OFF     # before every regression!
```

- Never run a regression with `WAVES=ON`: one GUI per test

Pitfalls

RTL Changes Are Not Detected

- The simulation step only depends on `rv64i.mem`, and `rv64i.mem` only on the `.asm` file
- After a change to the RTL (`.sv`), CMake rebuilds *nothing*: the old model is simulated again!
- Applies to both simulators (`obj_dir/Vtb_rv64i` and `compile.stamp`)

Before the next run:

```
rm -rf build                          # everything, or at least
rm -rf build/instr38ld/sim            # the simulation part of one test
```

Pitfalls

error.txt Accumulates

- The testbench opens `error.txt` with `"a"` (append): every run adds a line, old results stay

```
11/10/26 - ld: Test fail
11/12/26 - ld: Test ok
```

- `regression.txt` copies these files completely, so it also shows old results
- Delete old results before you judge them:

```
rm -f build/*/sim/error.txt
```

- Also: Verilator's exit code 1 after `$stop` is *not* a failure; only `error.txt` counts

Q&A

Questions?